

Diffusion Models

Diffusion models, the technique behind AI image generators such as Stable Diffusion, apply almost everything on this site. An SDE turns data into noise (Chapter 5). A *score* describes the noised data. A reverse-time SDE turns noise back into brand-new data. Here the "data" is one-dimensional: numbers drawn from two bumps. An image model does the same with every pixel at once.

7.1 Forward: Data to Noise

A diffusion model learns to create data by first watching how data is destroyed. Take the data samples (here, numbers clustered in two bumps) and let each one follow an SDE that slowly erases it into pure random noise.

Predict first: By $t = 3$, what will the histogram look like?

Still two bumps, just blurrier

A single standard bell curve, $N(0, 1)$

Flat

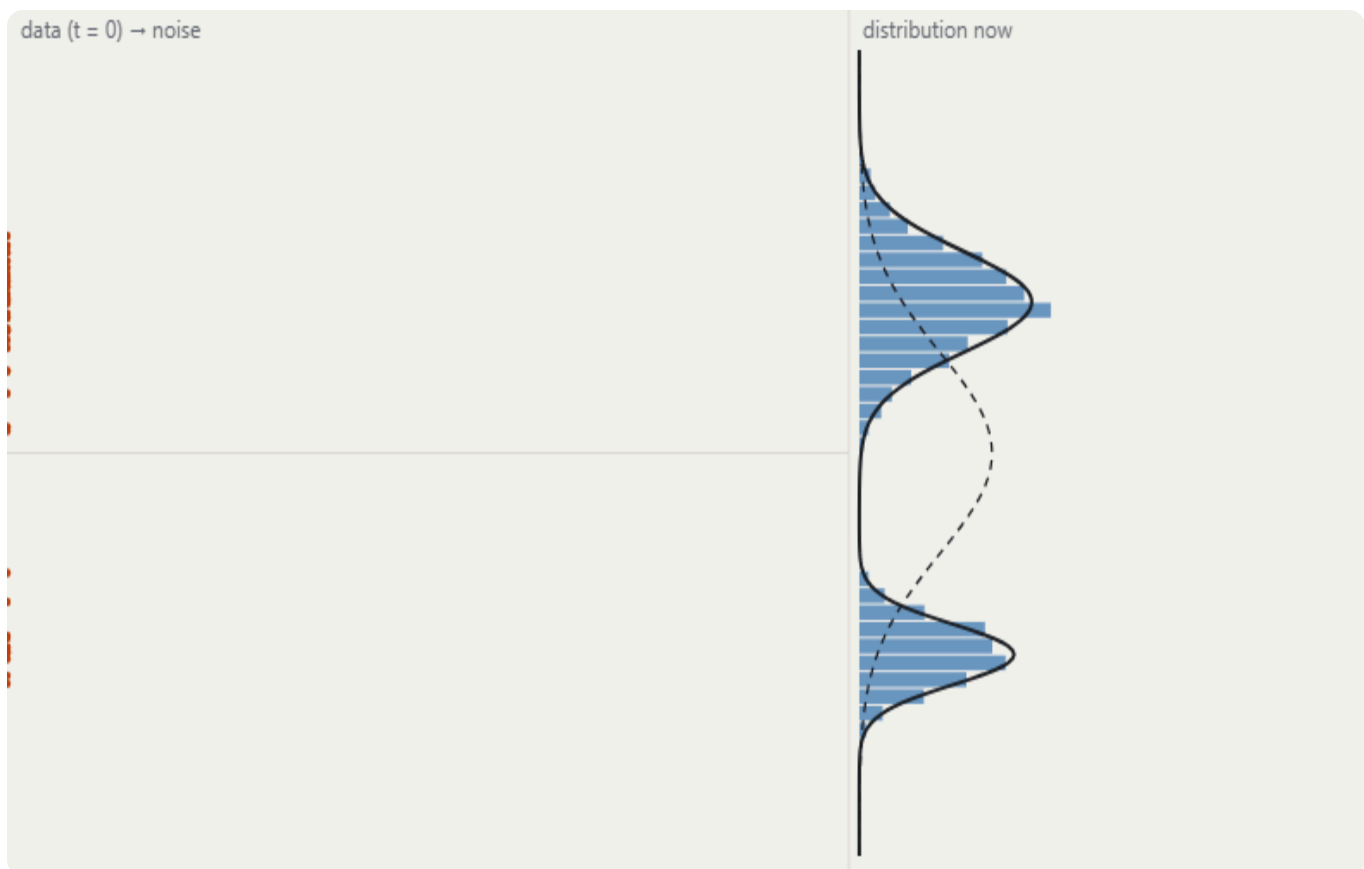
Answer: "A single standard bell curve, $N(0, 1)$ ". The drift shrinks every sample toward 0 by a factor e^{-t} , while the noise tops the spread back up toward 1. By $t = 3$ only $e^{-3} \approx 5\%$ of the original signal is left, so the data has become almost pure $N(0, 1)$ noise.

Scrub the time or press Play. The black curve is the exact distribution of the noised data. The dashed curve is pure noise.

$$dX_t = -X_t dt + \sqrt{2} dW_t \implies X_t = e^{-t} X_0 + \sqrt{1 - e^{-2t}} Z$$

- X_0 One data sample: a number from one of the two bumps.
- X_t The same sample after being noised for time t .
- $-X_t dt$ A drift pulling every sample toward 0. This is the Ornstein–Uhlenbeck process of section 5.2.
- $\sqrt{2} dW_t$ Random kicks. The $\sqrt{2}$ is chosen so the pull and the kicks balance at a long-run spread of exactly 1.
- Z A fresh standard bell-curve number, drawn from $N(0, 1)$.
- e^{-t} How much of the original sample survives: 37% at $t = 1$, 5% at $t = 3$.

In plain words: noising is "shrink a little, add a little noise", repeated. The formula on the right shows the result can be computed in one jump: keep a fraction e^{-t} of the sample and add bell-curve noise. After enough time, where a sample started no longer matters, and every dataset ends up as the same featureless $N(0, 1)$. That direction is easy. The useful direction is the reverse: start from noise and produce *new* data. That is what sections 7.2 and 7.3 build.



Left: 40 of the 3,000 samples being noised, up to the current time. Right: histogram of all samples now (blue), the exact noised distribution (black), and pure noise $N(0, 1)$ (dashed).

$t = 0.00$. Distance from pure noise $N(0, 1)$: 0.660 (1 = completely different, 0 = identical). Share of samples left of the valley: 35.5% (data: 35%, pure noise: 40.1%).

The forward process needs no learning: any data, however complicated, becomes the same $N(0, 1)$ noise. In an image model this happens to every pixel at once, and the end result looks like TV static.

Going deeper: the precise statement

The general forward SDE. Song et al. (2021) use the "variance-preserving" SDE $dX = -\frac{1}{2}\beta(t)X dt + \sqrt{\beta(t)} dW$ with a noise schedule $\beta(t)$. This page is the constant case $\beta = 2$. Its solution is $X_t = a_t X_0 + \sqrt{1 - a_t^2} Z$ with $a_t = e^{-\frac{1}{2} \int_0^t \beta(s) ds}$.

The discrete version (DDPM). Ho, Jain & Abbeel (2020) use steps $x_k = \sqrt{1 - \beta_k} x_{k-1} + \sqrt{\beta_k} z_k$, a discretisation of the same SDE (compare Euler–Maruyama, section 5.1). The idea of learning to reverse a noising process goes back to Sohl-Dickstein et al. (2015).

References: J. Sohl-Dickstein et al., *ICML* (2015). J. Ho, A. Jain & P. Abbeel, *NeurIPS* (2020). Y. Song et al., "Score-Based Generative Modeling through Stochastic Differential Equations", *ICLR* (2021).

Show the code (Python + NumPy)

```
import numpy as np
rng = np.random.default_rng(20)
w, m, s = np.array([0.35, 0.65]), np.array([-2.0, 1.5]), np.array([0.3, 0.5]) # the
"data": two bumps
k = rng.choice(2, size=100_000, p=w)
x0 = rng.normal(m[k], s[k])
for t in (0.0, 0.5, 1.0, 3.0):
    xt = np.exp(-t) * x0 + np.sqrt(1 - np.exp(-2 * t)) * rng.normal(size=x0.size) # exact
    noising to time t
    print(f"t = {t}: mean {xt.mean():+.3f}, spread {xt.std():.3f}, share left of the valley
    {np.mean(xt < -0.25):.3f}")
# pure noise N(0, 1): mean 0, spread 1, share left of -0.25 = 0.401
```

Output when we ran it:

```
t = 0.0: mean +0.281, spread 1.724, share left of the valley 0.349
t = 0.5: mean +0.166, spread 1.317, share left of the valley 0.363
t = 1.0: mean +0.099, spread 1.124, share left of the valley 0.375
t = 3.0: mean +0.014, spread 1.003, share left of the valley 0.397
```

7.2 The Score

To run the movie backward, one thing is needed at every noise level t and every position x : which direction makes a sample *more likely*, and how strongly. That is the *score*. The top panel

shows the noised density at the chosen t . The bottom panel shows its score, as a curve and as arrows.

Predict first: Drag t up to a lot of noise. What will the score become?

Zero everywhere

A straight line, $-x$, pointing everything to the centre

More and more jagged

Answer: "A straight line, $-x$, pointing everything to the centre". Pure noise $N(0, 1)$ has density proportional to $e^{-(x^2/2)}$, so $\log p = -x^2/2 + \text{a constant}$, and its slope is $-x$. Every arrow points to 0, and the further out it starts, the longer it is.

Drag the noise level. Near $t = 0$ the arrows point into each bump, and the valley between them pushes samples out toward one bump or the other.

$$s(t, x) = \frac{\partial}{\partial x} \log p(t, x) = \frac{\partial p(t, x) / \partial x}{p(t, x)}$$

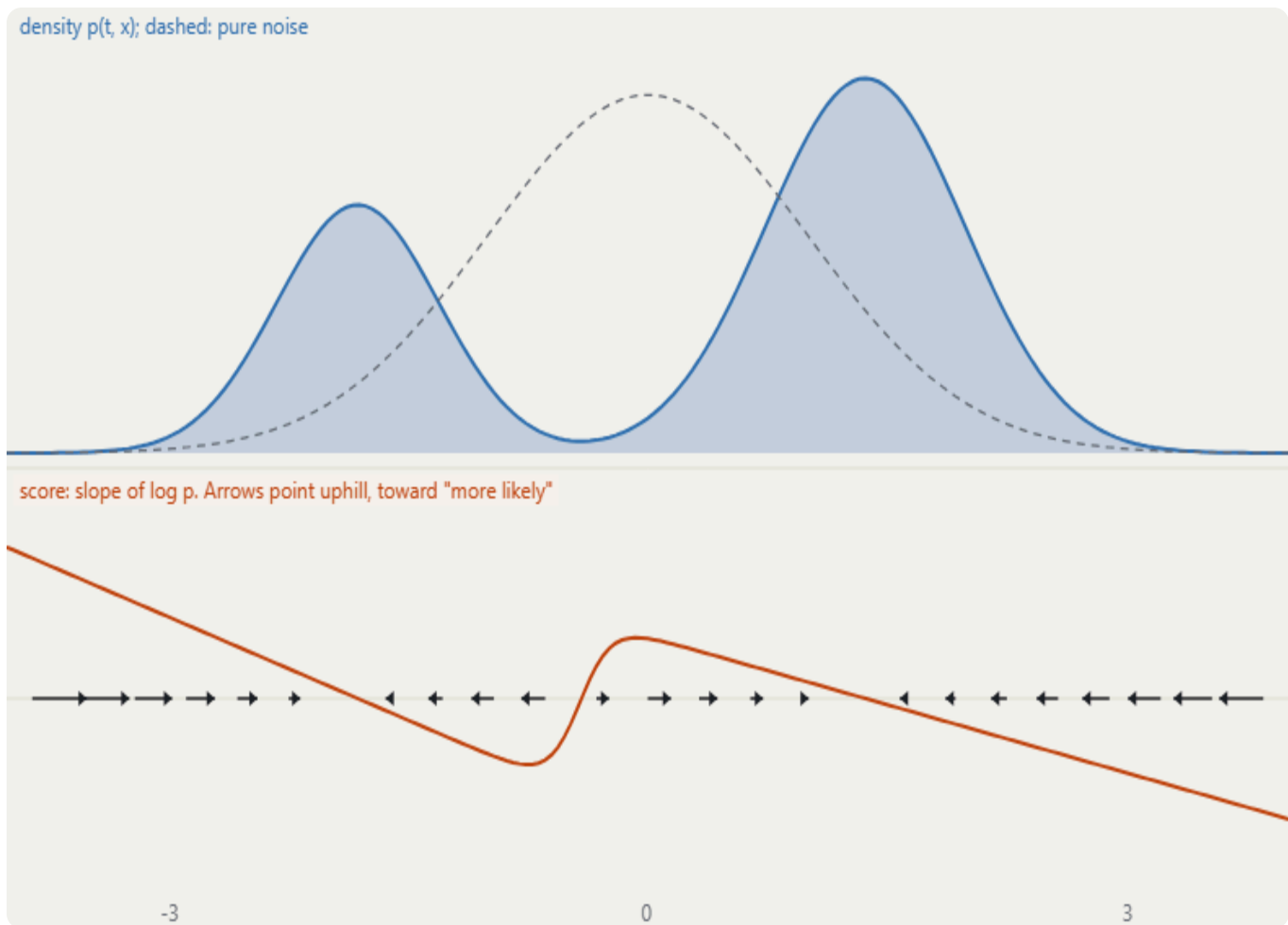
$p(t, x)$ The density of the noised data at time t (section 5.3): the black curve of section 7.1, and the blue curve here.

$\log p$ The natural logarithm of the density. Taking the log makes the score equally informative in crowded and sparse regions.

$\frac{\partial}{\partial x}$ The slope as x changes, with t held fixed (section 5.3).

$s(t, x)$ The *score*. Positive means "more likely to the right", negative means "more likely to the left", and its size says how steeply.

In plain words: the score is a compass pointing uphill on the density. Because it is the slope divided by the height, it stays strong even far out where the density is tiny: a lost sample always knows the way home. With little noise, the compass points into the nearest bump. With a lot of noise, everything looks like one bell curve and the compass simply points to 0. A real diffusion model does not know p . It trains a neural network to predict the score from noisy examples. Here the data is a known pair of bumps, so the score can be computed exactly.



$t = 0.10$. Score at $x = -3, 0, 3$: 4.67, 3.39, -4.26. Pure noise's score, $-x$, would give 3.00, 0.00, -3.00.

Knowing the score at every noise level is as good as knowing the data distribution itself. And a neural network can learn it from examples alone, by learning to *denoise*: the Going deeper panel explains why denoising and the score are the same thing.

Going deeper: the precise statement

Tweedie's formula: the score is a denoiser. For $X_t = aX_0 + \sqrt{1 - a^2} Z$ with $a = e^{-t}$: $\mathbb{E}[X_0 | X_t = x] = (x + (1 - a^2) s(t, x)) / a$. The best guess of the clean sample is the noisy one, nudged along the score and scaled back up.

Denoising score matching. A network $s_\theta(t, x)$ is trained to minimise $\mathbb{E} \left| s_\theta(t, X_t) + \frac{X_t - aX_0}{1 - a^2} \right|^2$, averaged over data X_0 , times t and noise. The target uses only the *known* noise that was added, yet the minimiser is the true score $s(t, x)$ (Vincent, 2011), a consequence of Tweedie's formula. This is how every diffusion model is trained.

References: A. Hyvärinen, *J. Machine Learning Research* 6 (2005), 695–709. P. Vincent, *Neural Computation* 23 (2011), 1661–1674. B. Efron, "Tweedie's formula and selection bias", *J. Amer. Statist. Assoc.* 106 (2011), 1602–1614.

Show the code (Python + NumPy)

```
import numpy as np
w, m, s = np.array([0.35, 0.65]), np.array([-2.0, 1.5]), np.array([0.3, 0.5])

def score(t, x):
    # d/dx log p(t, x) for the noised two-
    bump data
    mean, var = m * np.exp(-t), s ** 2 * np.exp(-2 * t) + 1 - np.exp(-2 * t)
    dens = w * np.exp(-(x - mean) ** 2 / (2 * var)) / np.sqrt(2 * np.pi * var)
    return np.sum(dens * -(x - mean) / var) / np.sum(dens)

for t in (0.05, 1.0, 5.0):
    print(f"t = {t}: " + " ".join(f"s({x:+.0f}) = {score(t, x):+.2f}" for x in (-3, 0,
3)))
print("pure noise N(0, 1) has score -x: +3, 0, -3")
```

Output when we ran it:

```
t = 0.05: s(-3) = +6.21  s(+0) = +4.43  s(+3) = -4.90
t = 1.0:  s(-3) = +2.62  s(+0) = +0.15  s(+3) = -2.73
t = 5.0:  s(-3) = +3.00  s(+0) = +0.00  s(+3) = -3.00
pure noise N(0, 1) has score -x: +3, 0, -3
```

7.3 Noise to Data: the Reverse-Time SDE

Now start from pure noise and run time backward. Anderson's theorem (1982) says the backward-running process is again an SDE, with one new ingredient: the score. Every particle starts as a random $N(0, 1)$ number at $t = 3$ and is steered, step by step, back to $t = 0$.

Predict first: Delete the score term from the reverse equation. What will the particles end up as?

The two bumps, just blurrier

One bump in the middle

A huge spread, far wider than the data

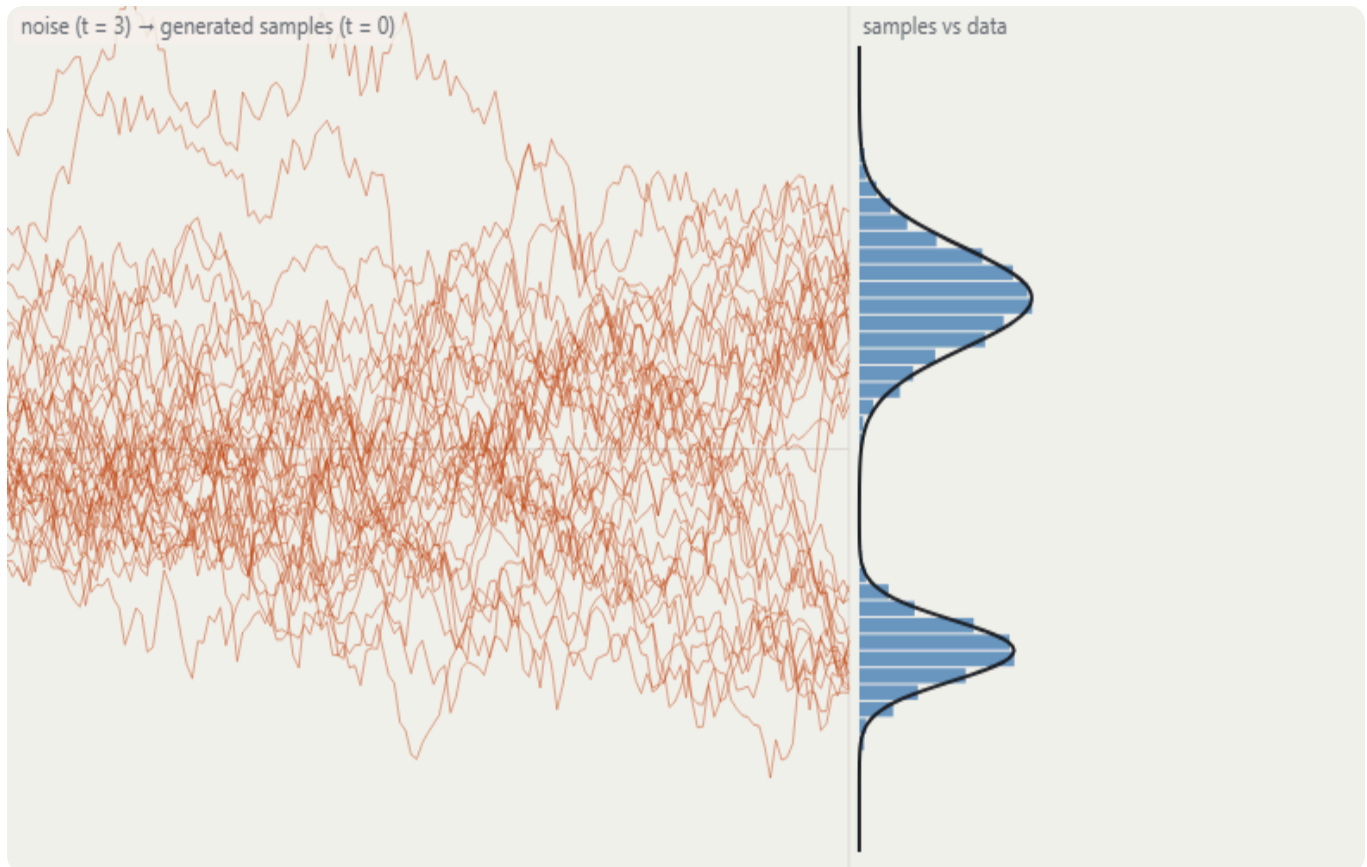
Answer: "A huge spread, far wider than the data". Without the score, the reversed drift pushes every particle away from 0 while the noise keeps adding spread, so the spread grows like e^t . The score term is the only thing that steers particles toward the data.

Choose a method. The paths run from noise (left, $t = 3$) to generated samples (right, $t = 0$). The histogram shows where all 3,000 particles end, against the true data (black).

$$dX_t = [-X_t - 2s(t, X_t)] dt + \sqrt{2} d\bar{W}_t \quad (\text{run with time going backward, } t : 3 -$$

time going backward	Each step moves t down by a small amount, so dt is negative.
$-X_t dt$	The forward drift of section 7.1. Run backward it pushes <i>away</i> from 0.
$-2s(t, X_t) dt$	The score correction. Because dt is negative, it moves each particle <i>along</i> the score arrows of section 7.2, toward more likely places.
$d\bar{W}_t$	Fresh random kicks for the backward run.
probability-flow ODE	The same equation with half the score term and no noise: $dX = [-X - s(t, X)] dt$. Each noise sample then maps to one data sample, with no randomness along the way.

In plain words: playing the forward movie backward naively goes wrong. The pull toward 0 becomes a push away from 0, and the noise keeps spreading. The score term is precisely the correction that makes the backward movie statistically identical to the forward one played in reverse. It gathers the noise into the right bumps, in the right proportions. The ODE version reaches the same distribution along smooth, deterministic paths, using the Fokker–Planck equation of section 5.3 to trade randomness for drift.



Share in the left bump: 36.4% (data: 35%). Bump centres -2.01 and 1.50 (data: -2 and 1.5); widths 0.31 and 0.49 (data: 0.3 and 0.5).

With the score, pure noise becomes new samples with the right bumps, in the right proportions (35% and 65%), at the right places and widths, without copying a single data point. This is how diffusion image generators draw a picture. They start from static and follow the reverse-time SDE (or its ODE) with a neural-network score, in millions of dimensions instead of one.

Going deeper: the precise statement

Anderson's theorem (1982). If $dX = f(t, X) dt + g(t) dW$ has densities p_t , then the same process run backward in time solves $dX = [f(t, X) - g(t)^2 \partial_x \log p_t(X)] dt + g(t) d\bar{W}$, where $\bar{W}$ is a Brownian motion running backward. Here $f = -x$ and $g = \sqrt{2}$, which gives the equation above.

Probability-flow ODE. The ODE $\dot{x} = f - \frac{1}{2}g^2 \partial_x \log p_t$ has the same densities p_t at every time. Substituting it into the Fokker–Planck equation (section 5.3) shows its drift term reproduces the diffusion term exactly. It enables fast deterministic sampling and exact likelihoods.

In practice. The score is a trained network. Sampling uses a few dozen to a thousand steps of an SDE or ODE solver, often in a compressed "latent" space (Stable Diffusion) and steered by a text prompt

("guidance").

References: B. Anderson, "Reverse-time diffusion equation models", *Stoch. Proc. Appl.* 12 (1982), 313–326.
Song et al., *ICLR* (2021). R. Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models", *CVPR* (2022).

Show the code (Python + NumPy)

```
import numpy as np
rng = np.random.default_rng(21)
w, m, s = np.array([0.35, 0.65]), np.array([-2.0, 1.5]), np.array([0.3, 0.5])

def score(t, x):
    # exact score of the noised data, for many x at once
    mean, var = m * np.exp(-t), s ** 2 * np.exp(-2 * t) + 1 - np.exp(-2 * t)
    dens = w * np.exp(-(x[:, None] - mean) ** 2 / (2 * var)) / np.sqrt(2 * np.pi * var)
    return np.sum(dens * -(x[:, None] - mean) / var, axis=1) / np.sum(dens, axis=1)

T, steps = 3.0, 600
h = T / steps
x = rng.normal(size=20_000) # start from pure noise
for i in range(steps): # run time backward from T to 0
    x = x + h * (x + 2 * score(T - i * h, x)) + np.sqrt(2 * h) * rng.normal(size=x.size)
left = x < -0.25
print(f"left bump {left.mean():.3f} (data 0.35); centres {x[left].mean():.2f}, {x[~left].mean():.2f} (data -2, 1.5)")
print(f"widths {x[left].std():.2f}, {x[~left].std():.2f} (data 0.30, 0.50)")
```

Output when we ran it:

```
left bump 0.352 (data 0.35); centres -2.00, 1.50 (data -2, 1.5)
widths 0.31, 0.50 (data 0.30, 0.50)
```

7.4 Langevin Sampling

The score can do something else: generate samples *without* the noise schedule. Let particles drift along the score of the data and add random kicks. In the long run their distribution settles on exactly the data distribution. This is *Langevin sampling*, and diffusion models use it as a building block. But "in the long run" hides a catch.

Predict first: Start every particle in the right-hand bump and run to $t = 10$. What share will be in the left bump?

About 35%, the right proportion

Close to 0%

About 50%

Answer: "Close to 0%". The valley between the bumps is so deep that particles almost never climb across it in the time shown, so almost none reach the left bump, even though 35% belong there in the long run.

Choose where the particles start, then scrub time or press Play. The black curve is the target: the two-bump data. The dashed line marks the valley between the bumps.

$$dX_t = s(X_t) dt + \sqrt{2} dW_t, \quad s(x) = \frac{d}{dx} \log \pi(x) \quad \implies \quad X_t \text{ is distributed :}$$

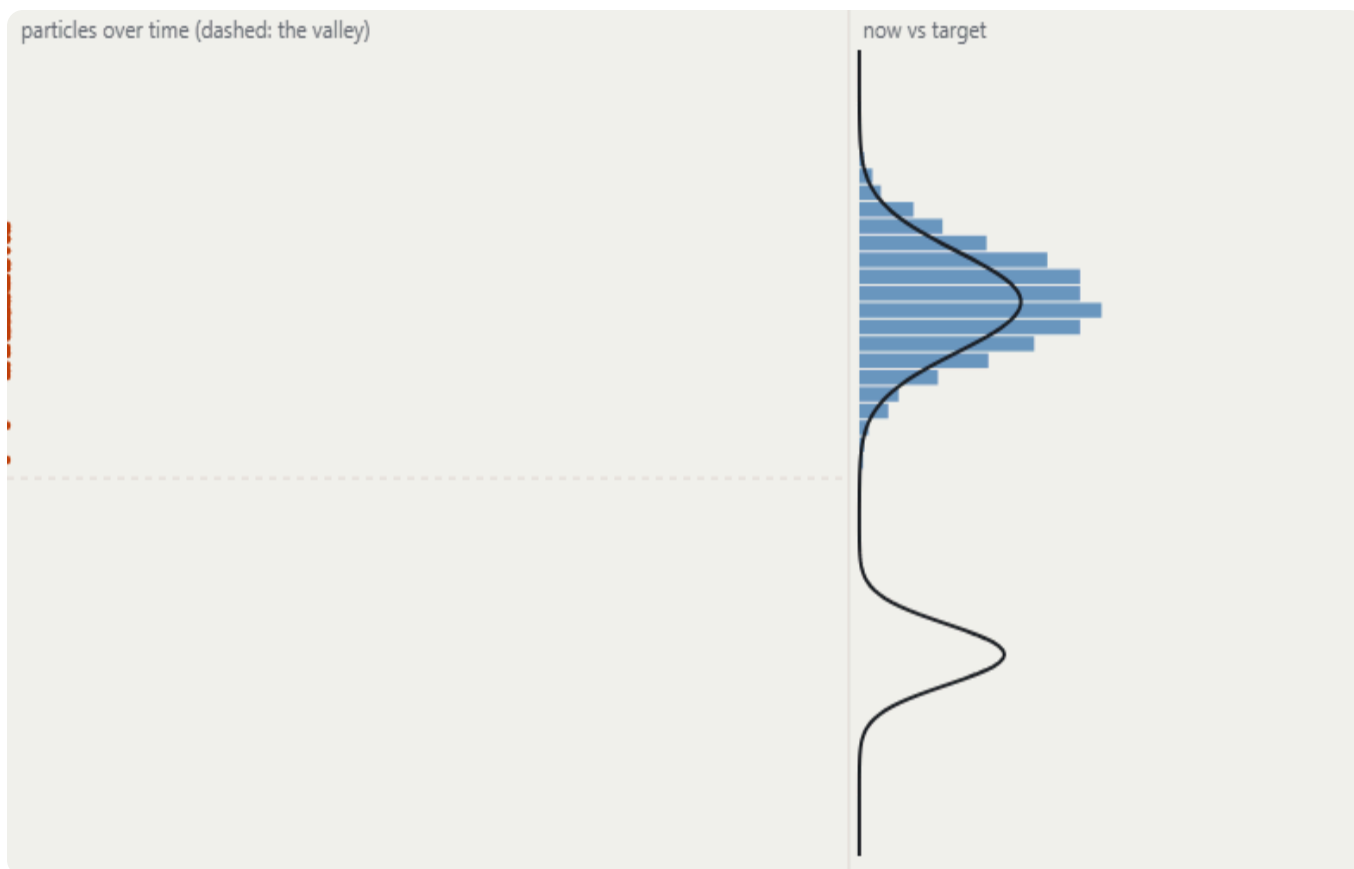
$\pi(x)$ The *target* density to sample from: here the two-bump data of this chapter.

$s(x)$ Its score: the slope of $\log \pi$, pointing toward more likely places (section 7.2 at noise level 0).

$\sqrt{2} dW_t$ Random kicks. The $\sqrt{2}$ balances the drift so that the resting distribution is exactly π .

$t \rightarrow \infty$ "In the long run". The theorem says nothing about *how* long, and that turns out to matter.

In plain words: the drift pushes each particle uphill on π and the noise spreads it out. The two balance exactly when the particles are distributed as π , which the Fokker–Planck equation of section 5.3 confirms. Within a bump this balance arrives quickly. But to move between bumps, a particle must climb against the score through a valley where π is almost zero, and that almost never happens. So the particles find the bumps' *shapes* fast, but keep whatever *proportions* they started with. Diffusion models avoid this trap by starting at high noise, where the valley is filled in (drag section 7.2's slider up), and lowering the noise gradually.



$t = 0.0$: 0.1% of particles are in the left bump (target: 35%). Particles that have ever reached the left bump: 0 of 2,000.

Langevin sampling reaches the right distribution eventually, but separated bumps can make "eventually" astronomically long: the bump shapes come out right, the proportions stay wrong. Running Langevin at a sequence of decreasing noise levels (*annealed* Langevin) fixes this, and it is the same idea as the reverse-time SDE of section 7.3.

Going deeper: the precise statement

Why π is the resting distribution. For $dX = s(X) dt + \sqrt{2} dW$ the Fokker–Planck equation can be written $\partial_t p = \partial_x (p \partial_x \log(p/\pi))$, which is zero exactly when $p = \pi$. Under mild conditions $p_t \rightarrow \pi$ from any start. The speed is governed by constants such as the spectral gap or a log-Sobolev constant.

Metastability. With $U = -\log \pi$, the expected time to cross from one bump to the other grows like $e^{\Delta U}$, where ΔU is the height of the barrier between them (the Eyring–Kramers law). A deep valley means an exponentially long wait.

In practice. The discrete update $x \leftarrow x + h s(x) + \sqrt{2h} z$ (the "unadjusted Langevin algorithm") is slightly biased for $h > 0$. Adding a Metropolis accept/reject step removes the bias (MALA). Song &

Ermon (2019) generated images with Langevin steps at a decreasing sequence of noise levels, the idea behind section 7.3.

References: G. Roberts & R. Tweedie, "Exponential convergence of Langevin distributions and their discrete approximations", *Bernoulli* 2 (1996), 341–363. A. Bovier et al., "Metastability in reversible diffusion processes I", *J. Eur. Math. Soc.* 6 (2004), 399–424. Y. Song & S. Ermon, "Generative Modeling by Estimating Gradients of the Data Distribution", *NeurIPS* (2019).

Show the code (Python + NumPy)

```
import numpy as np
rng = np.random.default_rng(22)
w, m, s = np.array([0.35, 0.65]), np.array([-2.0, 1.5]), np.array([0.3, 0.5]) # target:
the two bumps

def score(x):
    # d/dx log pi(x), for many x at once
    dens = w * np.exp(-(x[:, None] - m) ** 2 / (2 * s ** 2)) / s
    return np.sum(dens * -(x[:, None] - m) / s ** 2, axis=1) / np.sum(dens, axis=1)

dt, steps = 0.005, 2000 # run to t = 10
for start, x in (("all in the right bump", 1.5 + 0.5 * rng.normal(size=5000)),
                ("spread over [-4, 4]", rng.uniform(-4, 4, 5000))):
    for _ in range(steps):
        x = x + dt * score(x) + np.sqrt(2 * dt) * rng.normal(size=x.size) # Langevin step
    print(f"start {start}: share in left bump at t = 10 is {np.mean(x < -0.25):.3f} (target 0.35)")
```

Output when we ran it:

```
start all in the right bump: share in left bump at t = 10 is 0.004 (target 0.35)
start spread over [-4, 4]: share in left bump at t = 10 is 0.414 (target 0.35)
```

Chapter recap

Quick questions on this chapter's big ideas. Each answer is explained as soon as you choose.

1. After a long time, the forward noising process turns any data into...

The average of the data

Standard bell-curve noise, $N(0, 1)$

Exactly 0

Answer: "Standard bell-curve noise, $N(0, 1)$ ". The drift shrinks the data by $e^{(-t)}$ while the noise tops the spread up to 1, so every dataset ends as the same $N(0, 1)$.

2. At a noise level t and a position x , the score points...

Toward where samples are more likely

Always toward $x = 0$

Along the time axis

Answer: "Toward where samples are more likely". The score is the slope of $\log p$: it points uphill on the density. Only for pure noise does 'uphill' mean 'toward 0'.

3. What does the reverse-time SDE need that the forward process does not?

More noise

The score

A larger time step

Answer: "The score". Anderson's theorem: the backward process has an extra drift term, $-g^2 \times \text{score}$. Without it, noise stays noise.

4. Langevin sampling starts with every particle in the right-hand bump. Within the simulated time, what does it fail to do?

Find the shape of the right-hand bump

Move a fair share of particles into the other bump

Keep particles near any bump

Answer: "Move a fair share of particles into the other bump". The deep valley between the bumps is almost never crossed, so the proportions stay wrong. Diffusion models avoid this by starting from heavy noise, where the valley is filled in.